

说明

串行音频接口（SAI）和音频总线（I2S）均为同步串行音频通信外设，广泛用于数字音频的收发、编解码器连接、多通道音频路由及低延迟流处理。两者在操作层面均支持帧时钟（FS）和位时钟（SCK）驱动的串行数据搬移，I2S 作为一种专用协议，则更专注于标准双声道立体声传输，帧结构固定（左/右声道数据紧邻）；但 SAI 作为可配置的多功能外设模块，支持多种音频格式，具备双独立模块更加灵活。

适用范围

适用范围	系列
通用 MCU	CH32H417 等

目录

说明	1
目录	2
表格索引	2
第 1 章 I2S 简介	1
1.1 I2S 外设和 I2S 协议	1
1.2 I2S 外设支持的协议	1
1.2.1 飞利浦 I2S	1
1.2.2 MSB 对齐	1
1.2.3 LSB 对齐	1
1.2.4 PCM	2
第 2 章 SAI 简介	3
2.1 SAI 定义与结构	3
2.2 SAI 支持的协议与配置层次	3
2.3 SAI 字段对 I2S 标准的对应	3
2.4 I2S 外设 与 SAI 外设对比	3
第 3 章 SAI 外设实际例程	5
3.1 SAI 外设的初始化与配置	5
3.2 SAI 外设数据流程	6
历史版本	9
声明	10

表格索引

信号对应关系	1
SAI 对应关系	3
对比表格	3

第1章 I2S 简介

1.1 I2S 外设和 I2S 协议

I2S（Inter-IC Sound）由飞利浦提出，是在单一串行时钟上分时复用左右声道的三线同步音频协议。而 MCU 上的 I2S 一般来说是由 SPI 外设加上 MCLK 信号复用而来，具体对应关系如下，一般来说 I2S 外设不止支持 I2S 协议，还支持其他的类似协议。

表 1-1 信号对应关系

信号	I2S 功能	物理引脚	主模式	从模式
SD	串行数据	MOSI	输出	输入
WS	字选（声道指示）	NSS	输出	输入
CK	串行时钟	SCK	输出	输入
MCK	主时钟（可选）	独立引脚	输出，频率 $256 \times F_s$	不使用

1.2 I2S 外设支持的协议

一般来说 I2S 外设可以支持 4 种四种音频标准。

1.2.1 飞利浦 I2S

双声道立体声最常用的标准。

WS 语义：WS 对一帧的高/低半帧分别标记左右声道，是接收端区分左右声道的唯一依据。发送方在 CK 下降沿改变数据，接收方在 CK 上升沿采样，WS 与数据同步在下降沿跳变。

WS 相位：WS 在发送第一位（MSB）之前的 1 个位时钟就已有效，即 WS 领先数据 1 位。因为 WS 总在数据之前先稳定，上升沿采样才来得及判读当前半帧是左还是右声道。

WS 长度：恒等于半个帧——16 位帧时 WS 每半帧（每声道 16 个位时钟）翻转一次，32 位帧时每 32 个位时钟翻转；左右声道各占一个完整半帧。

波形示例（32 位帧）：WS 高电平期间发出右声道 32 位数据、低电平期间发出左声道 32 位数据；每个半帧的头部，先见到 WS 跳变、再隔 1 位时钟发出第一个（MSB）数据位。

1.2.2 MSB 对齐

WS 相位：WS 与数据的 MSB 同时产生——WS 不再领先 1 位，而是和每个半帧的第一位对齐；其余时序与飞利浦相同（MSB 先发、下降沿改数据、上升沿采样）。

WS 长度：同样恒为半个帧。

用法：要求 FS/WS 与数据首位严格对齐的 codec（部分老式 DAC/ADC）用此标准；它与飞利浦只差 1 个位时钟的 WS 偏移。

1.2.3 LSB 对齐

WS 相位：与 MSB 类似。

WS 长度：恒为半个帧，与飞利浦 / MSB 对齐相同——左右声道各占一帧的一个完整半帧。

对齐：有效数据靠帧尾（低位侧）对齐，即数据的 LSB 落在帧/槽的最后一个位时钟；这与 MSB 对齐首位即 MSB、数据靠帧头恰好对称。

用法：较少单独使用，多用于要求短收发延迟或按帧尾对齐接收的旧器件；在 I2S 侧主要作为兼容占位。

1.2.4 PCM

WS 语义：帧不含左右声道信息，此时 WS 改作同步脉冲（SYNC），只在每个数据字起始处给出一个标志位，不再区分左右，因此 CHSIDE 在 PCM 下无意义。适合独占链路上按字搬运串行编解码数据，而非立体声成对音频。

WS 长度：长帧同步脉冲固定 13 个位时钟，短帧仅 1 个位时钟。

数据与相位：每个数据字为固定 16 位，MSB 先发、紧随同步脉冲之后移出；一帧内只发这一个数据字，然后等待下一个同步脉冲。

第2章 SAI 简介

2.1 SAI 定义与结构

SAI (Serial Audio Interface) 是独立的专用音频外设，不占用其他外设的引脚。它由两个完全独立的音频子模块 A/B 组成，每个模块最多 4 个引脚 (SD、SCK、FS、MCLK)、各含一个 8 字 FIFO 和一个 32 位移位寄存器，经 IO 管理模块接到自己的引脚组。两个模块可配置为同步运行，以共用时钟与 FS、腾出引脚并实现全双工。

主/从、发/收发各模块可任意组合；单模块的收发取决于主/从与同步/异步设定，两模块同步时即可构成全双工。

2.2 SAI 支持的协议与配置层次

SAI 覆盖 I2S、LSB/MSB 对齐、PCM/DSP、TDM、AC'97，并可通过 PRTCFG 切到 SPDIF 输出与 AC'97 链路控制器模式。它用三组寄存器把"帧"、"帧内 Slot"、"数据/时钟"三层结构全部开放：

- 帧配置 FRCR：帧长 FRL[7:0] (8..256 个位时钟，帧长度 = FRL+1)、FS 有效长度 FSALL[6:0]、FS 极性 FSPOL、FS 偏移 FSOFF、FS 定义 FSDEF。
- Slot 配置 SLOTR：Slot 数 NBSLOT[3:0] (1..16)、Slot 大小 SLOTSZ[1:0] (=数据宽 / 16 / 32)、Slot 有效掩码 SLOTEN[15:0]、首位偏移 FBOFF[4:0]。
- 数据/时钟 CFGR1/CFGR2：数据位宽 DS[2:0] (8/10/16/20/24/32)、协议 PRTCFG、主从收发 MODE[1:0]、主时钟分频 MCKDIV、过采样 OSR、采样边沿 CKSTR、数据顺序 LSBFIRST、静音 MUTE、压扩 COMP、三态 TRIS、FIFO 阈值 FTH。

对 I2S 类协议，置 FSDEF=1 让 FS 兼作 SOF + 声道识别，此时 NBSLOT 须为偶数、左右声道均分帧；对 PCM/DSP/TDM/AC'97，置 FSDEF=0 使 FS 仅作 SOF。这套"帧 / Slot / FS 三者独立可配"的结构，是 SAI 用同一套硬件描述多种音频协议的机制。

2.3 SAI 字段对 I2S 标准的对应

相对 I2S 模块把模式取值"封装"成固定枚举，SAI 则更加灵活，把它们作为可直接写的字段，可以通过用户的需要自行配置相关的内容，故 SAI 还能描述 I2S 覆盖不到的自有帧结构（如 TDM 多槽位、非 2 的幂帧长、任意有效长度的 FS）。

表 2-1 SAI 对应关系

I2S 波形要素	对应 SAI 帧字段
飞利浦：WS 比数据提前 1 位	FSOFF=1 (FS 在 Slot0 第一位之前一位有效)
MSB 对齐：WS 与 MSB 同时	FSOFF=0 (FS 与 Slot0 第一位置位)
声道识别（左右均分帧）	FSDEF=1，NBSLOT 为偶数
PCM 长帧 WS 13 位 / 短帧 1 位	FSALL (有效电平长度 = FSALL+1)

2.4 I2S 外设 与 SAI 外设对比

表 2-2 对比表格

项目	I2S (复用自 SPI)	SAI (独立外设)
外设地位	SPI 的复用模式，占 SPI 引脚	独立音频外设，独立引脚组
子模块	单模块	两个独立音频子模块 A/B
数据通路	单个 16 位 寄存器	每模块 8 字 FIFO，一词一 Slot
声道/Slot	固定左右 2 声道	多种，可以自定义

数据位宽	16/24/32（24/32 需 2 次访问）	8/10/16/20/24/32
协议	飞利浦、MSB/LSB 对齐、PCM	多种可以自定义
主从	主/从	主/从×发/收任意组合，双模块可同步/异步
全双工	单工，一时刻单方向	双模块内部同步可全双工
帧同步	WS 相位/长度为固定枚举	FS 长度、极性、偏移、定义全可配
FIFO	无	有

第3章 SAI 外设实际例程

本章节描述了如何配置 SAI，使用 I2S 协议在 8KHz 的采样率下通过 DMA 发送数据的流程。

3.1 SAI 外设的初始化与配置

初始化时，首先基于系统时钟频率和所需的音频采样率，计算主时钟分频系数（MCKDIV）。计算公式为： $MCKDIV = (SYSCLK/6) / (SampleRate \times 256)$ ，其中分母中的 256 表示主时钟为采样率的 256 倍（ $MCLK = 256 \times FS$ ），该分频值决定了 SAI 模块生成的音频主时钟频率。

接着配置 SAI 工作模式与基本协议参数，具体分为以下几个步骤：

时钟分频与主从模式：使能分频链（SAI_MasterDivider_Enabled），将计算得到的 tmpdiv 写入主时钟分频器，使 SAI 模块作为主发送器（SAI_Mode_MasterTx）工作，由内部时钟源驱动总线，并主动输出位时钟和帧同步信号；

协议与数据格式：选择自由协议（SAI_Free_Protocol），允许用户自定义帧和时隙结构；数据长度设为 16 位（SAI_DataSize_16b），传输顺序为先发送最高有效位（MSB first），并在位时钟的上升沿对数据进行采样；

同步方式与 FIFO 阈值：配置为异步模式（SAI_Asynchronous），即 A 模块独立工作，不与 B 模块同步；发送 FIFO 阈值设为空（SAI_Threshold_FIFOEmpty），当 FIFO 为空时触发 DMA 请求，以连续填充数据。

然后配置音频帧结构，具体参数包括：

每帧包含 32 个位时钟周期（SAI_FrameLength = 32），其中帧同步信号有效持续 16 个位时钟周期（SAI_ActiveFrameLength = 16）；

帧同步信号仅在帧起始位置有效（SAI_FS_StartFrame），不用于声道识别；极性为高电平有效（SAI_FS_ActiveHigh），且帧同步信号提前于第一个数据位一个位时钟周期出现（SAI_FS_BeforeFirstBit）。

最后配置 Slot 参数，包括：

Slot 宽度等于数据宽度（16 位）；每帧包含 2 个 Slot（SAI_SlotNumber = 2），且 Slot 均被激活（SAI_SlotActive_ALL），确保所有 Slot 都能正常传输数据。

完成上述所有结构体配置后，分别调用 SAI_SlotInit 写入 Slot 寄存器，并调用 SAI_FlushFIFO 复位 FIFO 读写指针，清空可能残留的历史数据，确保发送起始状态干净一致。后续使能 SAI 模块即可开始音频数据的 DMA 或中断方式发送。

```
/* 采样率→主时钟分频：MCLK=256×FS，MCKDIV = (SYSCLK/6)/(FS×256)， */
const uint32_t tmpdiv = (RCC_ClocksStatus.SYSCLK_Frequency / 6) / (SampleRate * 256);

SAI_InitStructure.SAI_NoDivider      = SAI_MasterDivider_Enabled; /* 使能分频，MCKDIV 生效 */
SAI_InitStructure.SAI_MasterDivider = tmpdiv;                      /* 由 8kHz 采样率推导 */
SAI_InitStructure.SAI_AudioMode      = SAI_Mode_MasterTx;         /* 主发送器 */
SAI_InitStructure.SAI_Protocol       = SAI_Free_Protocol;         /* 自由协议，帧/Slot 自配 */
SAI_InitStructure.SAI_DataSize       = SAI_DataSize_16b;          /* 16 位数据 */
SAI_InitStructure.SAI_FirstBit       = SAI_FirstBit_MSB;          /* 先发 MSB */
SAI_InitStructure.SAI_ClockStrobing  = SAI_ClockStrobing_RisingEdge; /* 上升沿采样 */
SAI_InitStructure.SAI_Synchro        = SAI_Asynchronous;          /* 与 B 模块异步，不使用 B */
SAI_InitStructure.SAI_FIFOThreshold  = SAI_Threshold_FIFOEmpty;    /* 发送：FIFO 空触发 DMA */

/* 帧配置 */
```

```

SAI_FrameInitStructure.SAI_FrameLength      = 32;          /* 32 位时钟/帧 */
SAI_FrameInitStructure.SAI_ActiveFrameLength = 16;          /* FS 有效 16 位 */
SAI_FrameInitStructure.SAI_FSDefinition      = SAI_FS_StartFrame; /* FS 仅 S0F, 无声道识别 */
SAI_FrameInitStructure.SAI_FSPolarity        = SAI_FS_ActiveHigh; /* 高电平有效 */
SAI_FrameInitStructure.SAI_FSOffset          = SAI_FS_BeforeFirstBit; /* FS 提前于首 bit 一位 */

/* Slot 配置 */
SAI_SlotInitStructure.SAI_FirstBitOffset = 0;          /* FBOFF=0 */
SAI_SlotInitStructure.SAI_SlotSize       = SAI_SlotSize_16b; /* 槽宽=数据宽 */
SAI_SlotInitStructure.SAI_SlotNumber     = 2;          /* 2 个 Slot */
SAI_SlotInitStructure.SAI_SlotActive     = SAI_SlotActive_ALL; /* Slot 均有效 */
SAI_SlotInit(SAI_Block_A, &SAI_SlotInitStructure);

SAI_FlushFIFO(SAI_Block_A); /* FFLUSH: 复位 FIFO 读写指针, 清残留数据 */

```

3.2 SAI 外设数据流程

程序入口为 main 函数，进入正弦波数据生成阶段：根据预设的采样率（8000Hz）和音频频率（440Hz，即 A4 标准音高），循环计算每个采样点的正弦波幅值。每两个采样点为一组（左声道和右声道），左右声道数据相同，形成单声道双通道输出。幅值缩放为 1000，便于在 16 位有符号整数范围内表示。该数据将作为后续 DMA 传输的源数据缓冲区。

完成数据准备后，依次调用硬件初始化函数：

调用 SAI_Config，传入采样率参数，完成 SAI 模块的时钟分频计算、协议模式配置、帧结构定义及时隙参数设置，使 SAI 以主发送器模式准备就绪；

调用 SAI_GPIO_Config，配置 SAI 功能所需的引脚，将其设置为复用功能模式，以承载 SAI 总线的物理信号输出；

随后配置 DMA 通道用于音频数据的自动传输，具体步骤如下：

对指定 DMA 通道（DMA1_Channel1）进行复位，清除可能存在的历史配置状态；

配置 DMA 传输参数：外设地址设为 SAI 模块 A 的数据寄存器（DATAR），作为数据传输的目的地；内存地址设为预先计算好的正弦波数据数组 SAI_Data；传输方向为内存到外设

（DMA_DIR_PeripheralDST）；传输数据量为 DATA_SIZE 个半字（16 位）；外设地址固定不变（每次传输均写入同一数据寄存器），内存地址递增（顺序读取数组中的连续数据）；外设和内存数据宽度均为半字（16 位），与 SAI 配置的数据位宽一致；传输模式设为循环模式（DMA_Mode_Circular），使数据发送完毕后可自动从头开始循环播放，实现连续音频输出；DMA 通道优先级设为最高（VeryHigh），确保音频数据传输的实时性；禁止内存到内存模式，明确为外设传输场景；

调用 DMA_Init 将上述配置写入 DMA 硬件寄存器。

完成 DMA 基本配置后，通过 DMA_MuxChannelConfig 将 DMA 复用器通道 1 映射到 SAI 的 DMA 请求号（112），建立 DMA 请求与 SAI 模块之间的硬件连接。

最后依次使能各功能模块：

调用 DMA_Cmd 使能 DMA1_Channel1，使其进入待命状态；

调用 SAI_DMAMCmd 使能 SAI 模块的 DMA 发送请求，允许 SAI 在 FIFO 为空时自动触发 DMA 传输；

调用 SAI_Cmd 使能 SAI 模块，正式启动音频时钟和帧同步信号的输出，开始播放音频数据。

```

void SAI_Tx_DMA_Init(DMA_Channel_TypeDef *DMAy_Channelx, uint32_t pAddr, uint32_t mAddr,
                    uint16_t Length)
{
    DMA_InitTypeDef DMA_InitStructure = {0}; // DMA 配置结构体, 初始化为 0

    RCC_HBPeriphClockCmd(RCC_HBPeriph_DMA1, ENABLE); // 使能 DMA1 外设的 AHB 总线时钟

    DMA_DeInit(DMAy_Channelx); // 复位 DMA 通道, 清除历史配置状态
    DMA_InitStructure.DMA_PeripheralBaseAddr = pAddr; // 外设基地址: SAI 数据寄存器
    DMA_InitStructure.DMA_Memory0BaseAddr = mAddr; // 内存基地址: 音频数据数组
    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST; // 传输方向: 内存->外设
    DMA_InitStructure.DMA_BufferSize = Length; // 传输数据量 (单位: 个半字)
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable; // 外设地址不递增 (固定写 DATAR)
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable; // 内存地址递增 (顺序读取数组)
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord; // 外设数据宽度: 16 位
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord; // 内存数据宽度: 16 位
    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular; // 循环模式: 传输完自动重头开始
    DMA_InitStructure.DMA_Priority = DMA_Priority_VeryHigh; // 优先级: 最高, 保证音频实时性
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable; // 禁止内存到内存模式 (外设传输场景)
    DMA_Init(DMAy_Channelx, &DMA_InitStructure); // 将配置写入 DMA 硬件寄存器
}

void sai_paly(void)
{
    // 生成正弦波音频数据 (单声道双通道)
    // 音频频率: 440Hz (A4 标准音高), 采样率: 8000Hz
    for (int i = 0; i < DATA_SIZE; i += 2)
    {
        float t = ((float)i) / ((float)SAMPLE_RATE);
        SAI_Data[i] = 1000.0f * sinf(440.0f * TAU * t); // 左声道
        SAI_Data[i + 1] = SAI_Data[i]; // 右声道: 与左声道相同 (双声道同音)
    }

    SAI_Config(SAMPLE_RATE); // 配置 SAI 外设 (时钟分频、协议、帧结构、slot 等)
    SAI_GPIO_Config(); // 配置 SAI 功能引脚

    // 初始化 DMA 通道: 外设地址->SAI 数据寄存器, 内存地址->音频数据数组
    SAI_Tx_DMA_Init(DMA1_Channel1, (uint32_t)&SAI_Block_A->DATAR, (uint32_t)SAI_Data, DATA_SIZE);
    DMA_MuxChannelConfig(DMA_MuxChannel1, 112); // DMA 复用器映射: 将通道 1 映射到 SAI 的 DMA 请求号 112

    DMA_Cmd(DMA1_Channel1, ENABLE); // 使能 DMA 通道, 进入待命状态
    SAI_DMAMCmd(SAI_Block_A, ENABLE); // 使能 SAI 的 DMA 发送请求 (FIFO 空时自动触发 DMA 传输)

    SAI_Cmd(SAI_Block_A, ENABLE); // 使能 SAI 模块, 启动音频时钟和帧同步信号输出
}

```

```
while (1) // 主循环空转，所有音频数据搬运由 DMA 后台自动完成
{
}
}
```

历史版本

日期	版本	变更内容
2026/7/8	V1.0	初版发行

声明

本手册版权所有为南京沁恒微电子股份有限公司（Copyright © Nanjing Qinheng Microelectronics Co., Ltd. All Rights Reserved），未经南京沁恒微电子股份有限公司书面许可，任何人不得因任何目的、以任何形式（包括但不限于全部或部分地向任何人复制、泄露或散布）不当使用本产品手册中的任何信息。

任何未经允许擅自更改本产品手册中的内容与南京沁恒微电子股份有限公司无关。

南京沁恒微电子股份有限公司所提供的说明文档只作为相关产品的使用参考，不包含任何对特殊使用目的的担保。南京沁恒微电子股份有限公司保留更改和升级本产品手册以及手册中涉及的产品或软件的权利。

参考手册中可能包含少量由于疏忽造成的错误。已发现的会定期勘误，并在再版中更新和避免出现此类错误。